

Policy evaluation logic: how a request is actually decided

Domain 4 opens with the single highest-value module in this course.

There is no Allow that overrides a Deny.
Anywhere. Ever.

Six policy types

POLICY	ATTACHES TO	GRANTS OR CAPS
Identity-based	User, group or role	Grants
Resource-based	The resource itself	Grants, and names a principal
Service control policy	Org root, OU or account	Caps - principals
Resource control policy	Org root, OU or account	Caps - resources
Permissions boundary	A user or role	Caps that identity
Session policy	Passed at AssumeRole	Caps that session

ON THE EXAM

Only two of the six can grant anything. The other four can only take away, and silence in a cap is a denial.

The frame that makes it simple

- ◆ **Ceilings** – SCP, RCP, permissions boundary, session policy. They **intersect**. Every applicable ceiling must allow.
- ◆ **Grants** – identity-based and resource-based. They **union**. In the same account, either one is enough.
- ◆ **A ceiling that is silent about an action denies it.** There is no 'not mentioned, therefore fine'.
- ◆ **A ceiling can never add a permission** that no grant policy conferred. Attaching an SCP grants nobody anything.
- ◆ Effective permission = **intersection of every ceiling**, with **the union of the grants**, minus **any explicit deny**.

ON THE EXAM

Two categories, two behaviours. Get this and the order stops being memorisation.

The order AWS actually evaluates in

01

Deny?

Any matching explicit Deny in any policy type. Stops immediately.

02

Org ceilings

SCPs and RCPs. No applicable Allow means Deny.

03

Grants

Resource-based, then identity-based. One Allow is needed.

04

Local ceilings

Permissions boundary, then session policy.

05

Default

No Allow found anywhere is an implicit deny.

ON THE EXAM

Order between SCP and RCP does not matter, because both must allow. Order between denies and everything else matters enormously.

Service control policies

- ◆ Cap what **principals in your organisation** may do. They apply to users and roles in member accounts.
- ◆ **They never grant.** An option describing an SCP granting access is wrong, every time.
- ◆ **They do not restrict the management account root user.** They do apply to member account root users, as module 11 showed.
- ◆ **FullAWSAccess** is attached by default. Until you attach something narrower, nothing is capped.
- ◆ They do not apply to **service-linked roles**, so a service acting on your behalf is unaffected.

ON THE EXAM

An SCP is a fence around your own people. It has nothing to say about somebody in another organisation entirely.

Resource control policies

- ◆ Cap what may be done **to resources** in your organisation – by anyone, including principals outside it.
- ◆ An SCP cannot restrict a principal from another organisation. An RCP can.
- ◆ **RCPFullAWSAccess** is attached everywhere automatically and **cannot be detached**.
- ◆ Launch: S3, STS, KMS, SQS, Secrets Manager. **January 2026 added Cognito and CloudWatch Logs.**
- ◆ **July 2026:** quota raised to **2,000 RCPs per organisation.**

ON THE EXAM

SCPs and RCPs together build a data perimeter: my people cannot go out, outsiders cannot come in.

Permissions boundaries

- ◆ A managed policy attached to a **user or role** that caps what its identity-based policies can confer.
- ◆ Effective permissions are the **intersection**. Boundary allows S3 and EC2, identity policy allows S3 and IAM, result is S3 only.
- ◆ The classic use is **safe delegation**: let a team create roles, and require every role they create to carry your boundary.
- ◆ **A boundary alone grants nothing**. Attaching one to a role with no identity policy leaves it able to do nothing.
- ◆ Removing a boundary can **increase** what an identity may do. AWS documents this explicitly.

ON THE EXAM

How you delegate permission management without delegating unlimited permission.

Session policies

- ◆ Passed as a parameter at **AssumeRole** or federation. They cap that one session and nothing else.
- ◆ Also an intersection. A session policy can only **narrow** what the role already had.
- ◆ The pattern: one broad role, with an application passing a tight session policy per user or per tenant.
- ◆ They are invisible in the console's role view, which makes them a genuinely awkward thing to debug.
- ◆ The role's own trust policy and identity policy still apply. The session policy is an extra ceiling on top.

ON THE EXAM

Role has AdministratorAccess and the call is still denied? Suspect a session policy.

Resource-based policies, and the union

- ◆ Attached to the resource – bucket, queue, key, function – and they name a **Principal**.
- ◆ **Within one account, an Allow in either the resource or the identity policy is enough.**
- ◆ So a bucket policy can grant a user whose identity policy says nothing about S3.
- ◆ **A KMS key policy is the exception.** Without it allowing access, no IAM policy grants that key.
- ◆ Resource-based policies are also the only way to grant **across accounts**.

ON THE EXAM

The KMS key policy is the one resource policy that is not optional. Module 26 returns to it.

The subtlety almost nobody teaches

- ◆ Same account, resource policy granting directly to an **IAM user** or **session principal**: an implicit deny in the identity policy, boundary or session policy **does not change the outcome**.
- ◆ Resource policy naming an **IAM role ARN**: it **is** limited by an implicit deny in a boundary or session policy.
- ◆ Why: when a role is used, the principal making the request is the role **session**.
- ◆ Same behaviour whether you use **Principal** or **aws:PrincipalArn**.
- ◆ **An explicit deny still wins**. This exception concerns implicit denies only.

ON THE EXAM

User named: ceiling silence is irrelevant. Role ARN named: ceiling silence denies.

Cross-account: both sides must allow

- ◆ The caller's identity policy must allow it, and the owner's resource policy must allow it. No union across accounts.
- ◆ For a role: the **trust policy** in the target account, and **sts:AssumeRole** in the caller's policy.
- ◆ Ceilings do not all live in one account. The caller's SCP applies; so does the owner's RCP.
- ◆ A data perimeter therefore needs both: SCPs on principals, RCPs on resources.
- ◆ An **external ID** in the trust policy is the confused deputy defence. Module 21.

ON THE EXAM

The commonest cross-account failure: the resource policy was updated and the identity policy was not.

An IAM role in a single account has the AWS managed AdministratorAccess policy attached. A bucket policy on a bucket in the same account explicitly allows that role to call s3:GetObject.

The role also carries a permissions boundary that allows only ec2:* and cloudwatch:*. No SCP, RCP or session policy is involved. A principal assumes the role and calls s3:GetObject.

What is the result?

- ☐ A Allowed – AdministratorAccess grants it and the bucket policy confirms it
- ☐ B Allowed – the bucket policy grants directly, so the boundary does not apply
- ☐ C Denied – the permissions boundary does not allow s3, so it is an implicit deny in a ceiling
- ☐ D Denied – a permissions boundary always overrides a resource-based policy allow

◆ PAUSE AND DECIDE

What is the result?

- A

 Allowed – AdministratorAccess grants it
*Reads the grants and ignores the ceilings.
AdministratorAccess is a grant; the boundary caps what that grant may confer, and it is silent on s3.*
- C

 Denied – the boundary is silent on s3
Correct. The boundary is a ceiling, silence in a ceiling is an implicit deny, and a resource policy naming a role ARN is limited by that implicit deny.
- B

 Allowed – the bucket policy grants directly
The right exception applied to the wrong principal type. It holds for an IAM user or session principal named directly, not for a role ARN.
- D

 Denied – boundaries always override resource policies
Right answer, wrong rule. Boundaries do not universally override resource policies – the previous slide's user-versus-role distinction is exactly where that claim breaks.

THE SIGNAL

Options C and D reach the same verdict. Only one states the rule correctly, and the exam distinguishes between them.

In a single account, a bucket policy explicitly allows `arn:aws:iam::111122223333:user/analyst` to call `s3:GetObject` on one prefix. The user `analyst` has no identity-based policy attached at all, and belongs to no group.

There is no permissions boundary, no session policy, no SCP and no RCP. The user calls `s3:GetObject` on an object under that prefix.

What is the result?

- ☐ A Allowed – a resource policy granting directly to an IAM user is sufficient in the same account
- ☐ B Denied – the user has no identity-based policy, so there is nothing to allow the action
- ☐ C Denied – resource-based policies only work for cross-account access
- ☐ D Allowed – but only if the user is also granted `s3:GetObject` in an identity policy

◆ PAUSE AND DECIDE

What is the result?

- A

 Allowed – resource policy alone is sufficient same-account
Correct. Grants union within an account, so one Allow suffices – and here it is granted directly to the user, so no ceiling silence interferes.
- B

 Denied – no identity policy
Applies the cross-account rule to a same-account request. Both sides are required across an account boundary; within one account, either grant is enough.
- C

 Denied – resource policies are cross-account only
False. They are the only way to grant across accounts, and they work perfectly well within one.
- D

 Allowed only with a matching identity policy
The same misconception as B, expressed as a condition. Requiring both is the cross-account rule.

THE SIGNAL

Compare with scenario 1: the same union, one account, and the outcome flips because there a ceiling existed and here none does.

A role in account 111122223333 needs to read from a bucket in account 444455556666. The bucket policy in 444455556666 explicitly allows that role ARN to call s3:GetObject. The bucket is not encrypted with a customer managed key.

The role's identity-based policy in 111122223333 allows sts:AssumeRole and ec2:Describe* only. Calls to s3:GetObject on that bucket return AccessDenied.

Why, and what is the fix?

- ☐ A The bucket policy must use aws:PrincipalArn rather than the Principal element
- ☐ B An RCP in 444455556666 is blocking the request; remove it
- ☐ C The bucket must be in the same Region as the role for cross-account access
- ☐ D Cross-account needs both sides to allow; add s3:GetObject to the role's identity policy

◆ PAUSE AND DECIDE

Why, and what is the fix?

A Use `aws:PrincipalArn` instead of `Principal`

AWS documents both forms as behaving the same way. Swapping one for the other changes nothing about the outcome.

C Same Region required

Fabricated. Cross-account S3 access has no Region requirement, and the bucket policy would not have applied if it did.

B An RCP is blocking it; remove it

Nothing in the scenario mentions an RCP, and an RCP block would be an explicit deny. The identity policy's silence explains it without inventing anything.

D Both sides must allow; add `s3:GetObject` to the identity policy

Correct. Across an account boundary the union does not apply. The bucket policy allows it; the caller's identity policy is silent, which is an implicit deny.

THE SIGNAL

The most common cross-account failure in production: somebody updated the resource policy and stopped there.

A bank must guarantee that no principal outside its AWS Organization can access any S3 bucket in any of its 200 accounts, even if a bucket policy in one of those accounts allows an external account.

A previous attempt used an SCP denying s3:* when aws:PrincipalOrgID does not match. Buckets remained reachable by an external partner account that a team had granted in a bucket policy.

Why did the SCP fail, and what should the bank use?

- ☐ A The SCP was attached to an OU rather than the organisation root; reattach it at the root
- ☐ B SCPs govern principals in the organisation, not external ones; use an RCP on the resources
- ☐ C aws:PrincipalOrgID is not supported in SCPs; use aws:PrincipalOrgPaths instead
- ☐ D The SCP needs FullAWSAccess removed before a Deny statement takes effect

◆ PAUSE AND DECIDE

Why did the SCP fail, and what should the bank use?

A Reattach the SCP at the root

Placement changes which accounts are in scope, not which principals an SCP can govern. At the root it would still only reach the bank's own principals.

C aws:PrincipalOrgID unsupported in SCPs

False – it is one of the most commonly used condition keys in SCPs. The key works; the policy type is the problem.

B SCPs govern your principals; use an RCP

Correct. An SCP is a fence around your own identities. An RCP attaches to your resources and therefore constrains anyone touching them, including outsiders.

D Remove FullAWSAccess first

A real fact misapplied. FullAWSAccess matters for allow-list style SCPs; an explicit Deny takes effect regardless of what else is attached.

THE SIGNAL

This is precisely why RCPs exist. Deny statements in an SCP only ever reach principals the SCP applies to.

An engineer assumes a role with AdministratorAccess through an internal tool and receives AccessDenied on kms:Decrypt. The same role, assumed directly with the AWS CLI, works.

There is no SCP or RCP denying KMS, no permissions boundary on the role, and the KMS key policy allows the role. CloudTrail shows the denied call was made by the same role ARN in both cases.

What is the MOST likely cause?

- ☐ A The key policy allows the role but not the role session
- ☐ B The internal tool is assuming the role in a different Region
- ☐ C The internal tool passes a session policy that does not allow kms:Decrypt
- ☐ D AdministratorAccess excludes KMS actions by design

◆ PAUSE AND DECIDE

What is the MOST likely cause?

- A

 Key policy allows the role but not the session
A key policy naming a role ARN covers sessions assumed from it. The scenario also says the CLI path works with the same key policy.
- B

 Different Region
Would produce a not-found or a different key entirely, and the scenario says the key policy allows the role. It also does not explain a permissions-shaped error.
- C

 The tool passes a session policy that omits kms:Decrypt
Correct. Same role, two paths, one fails – the difference is what the caller passes at AssumeRole. A session policy caps the session and is invisible on the role.
- D

 AdministratorAccess excludes KMS
False. AdministratorAccess is Allow on all actions and all resources. KMS access can still fail via the key policy, but not because of that managed policy.

THE SIGNAL

Same identity, two paths, different outcomes. The difference is never in the role – it is in what the caller passed.

Six things to carry forward

- ◆ **There is no Allow that overrides a Deny.** Scan for explicit denies before anything else.
- ◆ **Ceilings intersect, grants union.** SCP, RCP, boundary and session cap; identity and resource grant.
- ◆ **Silence in a ceiling is a denial.** Silence in a grant just means the other grant has to carry it.
- ◆ **Same account: either grant suffices. Cross-account: both are required.**
- ◆ **SCPs fence your principals. RCPs fence your resources,** including against outsiders.
- ◆ **Resource policy naming a user ignores ceiling silence. Naming a role ARN does not.**

ON THE EXAM

Next: reading and writing IAM policies – conditions, condition keys, and the wildcards that betray you.